

AMLGuard

Manual de Estudo - Volume 4

MLflow Tracking, Model Registry, Signature e Promocao de Versoes



Objetivo deste volume

Explicar, com linguagem simples e conexao com o projeto real, como o AMLGuard passou de um arquivo de modelo salvo localmente para um modelo rastreado, versionado, carregavel por alias e protegido por um contrato de entrada e saida.

Este volume cobre as entregas dos Dias 13 e 14: MLflow Tracking com SQLite, registro de metricas e artefatos, empacotamento PyFunc, signature, input example, Model Registry e promocao candidate -> champion.

Versao do projeto: 0.1.0 | Modelo: XGBoost - Scale Pos Weight | Threshold publico: 0.892163

1. Do experimento ao produto versionado

No treinamento tradicional, o resultado pode terminar como um arquivo `model.joblib`. Esse arquivo contem o pipeline treinado, mas sozinho não responde perguntas importantes: qual experimento o gerou, quais métricas foram obtidas, quais colunas ele aceita, qual formato ele retorna e qual versão está aprovada para uso?

O MLflow separa essas responsabilidades em duas camadas:

Camada	Pergunta que responde	Evidencia no AMLGuard
Experiment Tracking	Como essa execução foi produzida?	Parâmetros, métricas, tags, artefatos e Run ID
Model Registry	Qual versão está aprovada para uso?	Version 1, aliases candidate e champion, signature

Analogia: laboratório e linha de produção

O Tracking é o caderno do laboratório: registra como o teste foi feito. O Registry é o estoque controlado da fábrica: guarda versões imutáveis e indica qual está liberada para operação.

Lição central: uma run e um registro de experimento; uma model version e um artefato preparado para ser carregado e utilizado.

Autoteste 1

1. Uma run verde significa automaticamente que o modelo está aprovado para produção?
2. O Model Registry substitui o histórico de experimentos?
3. Um arquivo `joblib`, sem signature, documenta sozinho o contrato de entrada e saída?

Respostas: não; não; não.

2. O que foi registrado no Dia 13

O AMLGuard passou a usar um backend SQLite local, em `mlflow.db`. O banco guarda a estrutura de experimentos, runs, parametros, metricas, tags e referencias de artefatos. A interface local consulta esse mesmo banco.

Fluxo implementado



Principais grupos de informacao

Grupo	Exemplos	Por que importa
Parameters	random_state, test_size, scale_pos_weight, xgb_*	Reproduzir a configuracao
Metrics	Average Precision, recall, precision, threshold, FP/FN/TP/TN	Comparar desempenho e operacao
Tags	git_commit, baseline_gate, model_version	Governanca e busca
Artifacts	modelo, metadata, baseline, config, evaluation	Auditoria e evidencia

Erro real: FileStore em maintenance mode

A primeira tentativa usou o backend de arquivos legado. A versao atual do MLflow bloqueou esse caminho por padrao. A correcao profissional foi migrar o backend para SQLite, em vez de apenas ativar uma variavel para continuar usando o modo antigo.

Erro real: metadata sem accuracy

O arquivo de metadata havia sido produzido por uma versao anterior do codigo e nao possuia o campo accuracy. O tracking foi ajustado para aceitar metadados legados e derivar accuracy a partir da matriz de confusao: $(TP + TN) / \text{total}$.

Licao central: metadados tambem possuem versoes. Codigo novo deve tratar, validar ou migrar formatos antigos de forma explicita.

3. O modelo MLflow do Dia 14

O modelo foi empacotado como MLflow PyFunc. O wrapper carrega o pipeline sklearn persistido, calcula `predict_proba` e retorna o mesmo contrato utilizado pela camada de serving.

Contrato de entrada

Feature	Tipo no exemplo	Significado
Payment Format	string	Formato do pagamento
Amount Paid	float64	Valor pago
sender_previous_tx_count	int64	Historico anterior do remetente
is_business_hours	int64	Indicador de horario comercial
same_account	int64	Origem e destino representam a mesma conta

Contrato de saida

Campo	Tipo	Regra
risk_score	float	Probabilidade estimada de lavagem
is_alert	bool	risk_score >= 0.892163
threshold	float	Threshold publico congelado
model_version	string	Versao da aplicacao: 0.1.0

Por que a signature importa

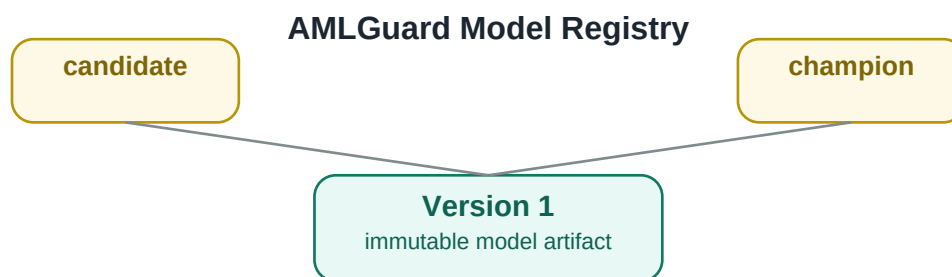
A signature e o contrato formal do modelo. Ela permite ao MLflow conhecer a estrutura esperada da entrada e da saida. O input example complementa esse contrato com dados concretos, usados durante a validacao do empacotamento.

O warning sobre colunas inteiras

O MLflow avisou que colunas inteiras nao representam valores ausentes. Isso nao bloqueou o modelo porque essas features sao obrigatorias. Para um sistema que aceite valores ausentes, seria necessario representar essas colunas como float ou definir tratamento explicito de missing values.

Licao central: warnings nao devem ser ignorados nem tratados automaticamente como falhas. Primeiro se identifica o risco real; depois se decide se ele afeta o contrato atual.

4. Candidate, champion e rollback



Aliases move. The registered version does not change.

Uma versao registrada e imutavel. Os aliases sao referencias moveis que apontam para uma versao. O AMLGuard usa dois aliases:

Alias	Significado	Momento de uso
candidate	Versao empacotada, testada e aguardando aprovacao humana	Depois do smoke test e antes da promocao
champion	Versao oficial aprovada para consumo	Depois da revisao do candidate

Promocao implementada

Antes de mover o alias champion, o comando verifica se o candidate possui `validation_status=passed`, carrega o modelo pelo alias e executa uma previsao. A promocao nao cria uma nova versao e nao retreina o modelo.

Rollback

Se uma versao futura apresentar problema, champion pode voltar a apontar para uma versao anteriormente validada. O historico nao deve ser apagado: ele e parte da trilha de auditoria.

Autoteste 2

1. Promover candidate para champion duplica o modelo?
2. O alias champion deve ser codificado como um numero fixo no consumidor?
3. Rollback exige retreinamento?

Respostas: nao; nao; nao.

5. Evidencias e comandos essenciais

Registrar uma run existente

```
python -m src.models.tracking
```

Iniciar a interface local

```
python -m mlflow server --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5000
```

Empacotar e registrar candidate

```
python -m src.models.mlflow_model
```

Promover candidate para champion

```
python -m src.models.mlflow_model --promote-candidate
```

Carregar por alias

```
model = mlflow.pyfunc.load_model('models:/AMLSGuard@champion')
```

Definicao de pronto dos Dias 13 e 14

Evidencia	Status
Run com parametros, metricas, tags e artefatos	Concluido
Backend SQLite e UI local funcionando	Concluido
Modelo registrado como AMLGuard Version 1	Concluido
Signature de 5 entradas e 4 saidas	Concluido
Alias candidate e smoke test por recarga	Concluido
Promocao para champion e inferencia por alias	Concluido
56 testes e Ruff limpo	Concluido

Resumo para entrevista

"No AMLGuard, implementei rastreamento de experimentos com MLflow e SQLite, registrando parametros, metricas, tags e artefatos. Depois empacotei o pipeline como PyFunc com signature e input example, registrei uma versao no Model Registry e implementei promocao controlada por aliases candidate e champion, incluindo smoke test e validacao de paridade com o pipeline local."

Resultado profissional

O projeto agora demonstra nao apenas modelagem, mas tambem reproducibilidade, governanca, contrato de inferencia, versionamento e promocao segura de modelos - competencias centrais de MLOps aplicado.